Realcryptik: Autonomous Election Verification with Zeeperio

Anonymized for review

Abstract. Many cryptographic end-to-end election verification (E2E-V) protocols allow the public to verify proofs, but participation in this important process is largely left as an opt-in, adhoc exercise. We present Zeeperio, a modern update to the Eperio protocol toward making election proof verification routine and automatic. Zeeperio is a zk-SNARK-based system that verifies proofs automatically via smart contracts, improving on Eperio's linear-complexity through a novel, succinct argument with size and verification time constant in the number of ballots.
We implemented Zeeperio and tested it on election with $100,000$ ballots and 5 candidates and demonstrated verification on Ethereum L1 for under $60USD (at time of writing), using precompiles introduced in EIPs 196 and 197. By comparison to a leading STARK-based implementation, which required proofs in the Megabyte range for $32,000$ ballots and took 9 days to generate, Zeeperio's proof for $100,000$ ballots measures in the kilobyte range and was completed in 3 hours on a standard CPU. These orders-of-magnitude improvements to performance directly translate to lower transaction costs and improved on-chain scalability for larger elections, toward making routine, automated election verification and interesting and practical possibility.

Keywords: end-to-end verifiable voting, zero-knowledge proofs, zk-SNARKs, polynomial interactive oracle proofs, smart contracts

1 Introduction

Decades of research on end-to-end verifiable (E2E-V) elections have shown that strong cryptographic guarantees can protect the integrity of elections and the secrecy of voter ballots [17]. In recent years, pilot projects and deployments of E2E-V platforms have also demonstrated that these systems are usable and scalable to an appropriate degree [2, 9, 4, 13]. Despite these efforts, some governments continue to conduct elections without meaningful verification or openness to scrutiny, in which voter ballot selections and identities are stored together within the election service (see, e.g., Cardillo et al. [10]).

In this paper, we contribute to a broader discussion the cryptographic voting community has been having for some years: do we compromise our ideals and offer watered-down systems that the industry might find more palatable if they offer tangible security advantages over the status quo? We relate this to the term realpolitik, which describes statecraft based on pragmatic calculation rather than

ideology, and propose *realcryptik* to describe a similar pragmatic approach in the election verification context.

Flavours of realcryptik already permeate the research literature. Despite opposition to internet voting, Helios [1] is perhaps the most widely used E2E-V voting technology. While researchers argue that coercion resistance is essential for internet voting, Helios provides none. Scantegrity II [11] was the first E2E-V system to see governmental use. It is built around the compromise that, in a paper ballot system, it is acceptable for an optical scanner to learn how the voter voted, sidestepping the complex ballot mechanics of its predecessors to avoid this. In recent years, serious discussions have arisen around whether E2E-V should be largely abandoned in favor of risk-limiting audits. Should we affirm the use of ballot marking devices when we know most voters do not carefully verify their outputs [5]?

We propose a different compromise in this paper: Allow election service providers (vendors) to continue collecting unencrypted ballot selections while introducing a measure of verifiability. For the election authorities (EAs), our approach continues to rely on legal protections implemented through procedures, i.e., without formal cryptographic secrecy protections and guarantees.

The research question of this paper is how to modify the cryptography of an E2E-V voting system to provide integrity without requiring the EA to maintain ballot secrecy (while still providing receipt-freeness otherwise). The simpler threat model allows us to streamline the cryptography. In particular, we can prove that encrypted (or committed, in our case) ballots are tallied correctly, rather than computing the tally under encryption. This shifts the protocol away from multiparty computation and threshold cryptography toward an all-encompassing zero-knowledge proof (ZKP). While ZKPs have been used in voting systems since their invention in the 1980s, we focus on modern succinct ZKPs called zk-SNARKs.

Zeeperio. Zeeperio is a modernization of the Eperio E2E-V backend [14]. Eperio was an attempt to simplify the cryptography of Scantegrity II by making larger, aggregated commitments that could be implemented with file-based encryption, enabling more practical implementations of the proof verification using pre-existing and commonplace software. For example, the paper demonstrated an implementation of the verifier using only the built-in features of Microsoft Excel. Eperio's security was studied in detail using formal methods in a CCS 2021 paper [3]. Like Scantegrity (and related schemes of the era), Eperio still required election authorities (EAs) to organize an independent and explicit proof-checking effort to ensure *someone* checked the proof at all.

If, however, verification could be reduced to a succinct (zk-SNARK) argument, voters and candidates can still independently verify the election (if they choose), but the proof could now be automatically checked by a smart contract, giving EAs automatic assurance that at least *someone* checked it. In this paper, that someone is the Ethereum network, which ties the acceptability of the election proof to the network's $100+B USD market capitalization. The design

challenge of this work is to fit the proof within the highly restricted computational limits of the smart contract setting.

1.1 Contribution

Zeeperio is a modernization of the Eperio election-verification approach, replacing antiquated cut-and-choose full-open commitments with modern, succinct knowledge arguments for automated verification in a smart-contract setting:

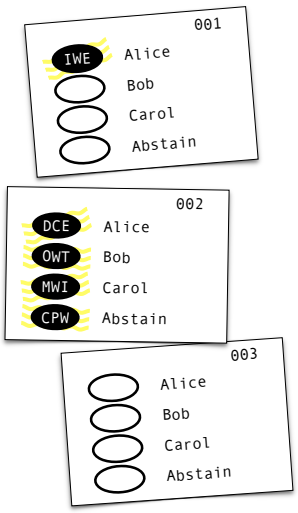
1. **Succinct universal verification:** Election constraints are encoded in a single polynomial interactive proof (PIOP) using a zkSNARK approach in Plonk [15] with a size and runtime constant in the number of voters.
2. **Automatic verification:** The succinctness of Zeeperio arguments enables automatic verification in a smart contract setting, reducing the need for EAs to coordinate explicit independent verification efforts (as in Eperio).
3. **Implementation:** We implement the Zeeperio knowledge arguments and demonstrate their performance in an Ethereum smart contract, showing that an election with $\mathcal{B} = 100,000$ ballots and $\mathcal{C} = 5$ candidates can be verified on-chain for under $60 USD.
4. **Enhanced dispute resolution**: As a result of confirmation code commitments made within a single polynomial, Zeeperio improves the dispute-resolution process by enabling a succinct proof that a disputed code is incorrect without revealing any other codes or the voter's selections.

2 Background and Related Work

2.1 Eperio

We assume the reader is already familiar with end-to-end verifiable (E2E-V) voting protocols, which allow for: (i) an election authority to prove that an election tally is computed correctly without learning individual voter choices, and (ii) voters to check that their ballots are included unmodified in the tally without revealing how they voted to anyone else. Eperio is an E2E-V tallying protocol (or 'backend') that works with several styles of paper ballots, including Pret a Voter and Scantegrity I. A brief description of Eperio is provided in Figure 1 with a Scantegrity II-style ballot. In addition to succinctness, we also clean up a few issues that were not fully resolved in Scantegrity II and Eperio:

- As pointed about by Basin *et al.* [3], Eperio does not prove ballots were not overvoted; Zeeperio does (§3.7).
- As acknowledged by Essex *et al.* [14], Eperio does not support Scantegrity II ballots; Zeeperio does (§4.2).
- Scantegrity II [11] supports dispute resolution, but requires all confirmation codes on the ballot to be revealed. Zeeprio offers a simple proof of an incorrect code without revealing information about all codes (§4.3).



Index	Ballot	Position	Code	Audit	Mark	Selection
0	001	1	IWE	0	1	Alice
1	001	2	XJS	0	0	Bob
2	001	3	NIP	0	0	Carol
3	001	4	QEM	0	0	Abstain
4	002	1	DCE	1	0	Alice
5	002	2	OWT	1	0	Bob
6	002	3	MWI	1	0	Carol
7	002	4	CPW	1	0	Abstain
8	002	1	WNJ	0	0	Alice
9	003	2	IES	0	0	Bob
10	003	3	CNE	0	0	Carol
11	003	4	PCS	0	0	Abstain
12	Pad		0	0	0	
13	Pad		0	0	0	
14	Pad		0	0	0	
15	Pad		0	0	0	

Fig. 1. An Eperio/Zeeperio table illustrated with Scantegrity II ballots that reveal a hidden confirmation code for each marked selection. Eperio commits to columns: ballot, position, mark, selection (it technically does not work with Scantegrity II, so selection encodes a Scantegrity I or Pret a Voter layout). Many copies of the table are made, shuffled row-wise, and, based on coin tosses, the EA opens either {ballot, position, mark} or {mark, selection}, enabling receipt checks and tally checks, respectively. In Zeeperio, the table is not shuffled, so the white columns are public information and not committed to. The pink columns are padded (described below) and committed to and the EA uses these commitments to demonstrate certain constraints hold for column.

2.2 Succinct Proofs

While zero-knowledge proofs/arguments (ZKPs) date back to the 1980s [16, 7], classic ZKPs produce proofs that are at least linear in the size of the proof statement. By contrast, succinct zero-knowledge arguments (SNARKs) are sublinear, with sizes typically logarithmic or constant. A Polynomial Interactive Oracle Proof (PIOP) is a pattern shared by several SNARKs [21, 12, 8] where the prover specifies polynomials and is challenged to open them at random points. PIOP does not say *how* to make a SNARK; it is merely a generic pattern.

To realize a PIOP, the prover must send a succinct representation of each polynomial to the verifier, which commits the prover to those polynomials. When opening a polynomial, the verifier must be certain it is the opening of the same polynomial that was committed to. PIOPs rely on a cryptographic primitive called a polynomial commitment scheme (PCS) to handle this. The rest of a PIOP protocol does not use any cryptography. A number of PCSs have been proposed; the most prominent are KZG (very short, based on bilinear pairings, requires a trusted setup) and (DEEP-)FRI (longer, based on hash functions,

conjectured to be post-quantum secure, no trusted setup). In Zeeperio, we use KZG [20] because it integrates well with Ethereum.

KZG uses a universal trusted setup: a secret value τ is chosen at random, used to compute a structured reference string (SRS) $(g^{\tau}, g^{\tau^2}, g^{\tau^3}, \ldots)$ in a pairing-friendly group and then erased. Knowledge of τ after the setup is sufficient to fake KZG openings and forge false ZKP transcripts. However, the setup can be run as an open participation multiparty computation, in which participants prove that their contributions are correct, and only one honest participant must forget τ for the SRS to be secure. Ethereum ran an open trusted setup in 2023, accepting 100K contributions over 208 days. A single SRS, such as this one, can be shared by anyone who wishes to make a KZG commitment.

In practice, zk-SNARKs for a specific computation can be deployed in different ways: (a) written in a high-level language and compiled to VM instructions, (b) written in special markup languages and compiled into arithmetic circuits, or (c) written as a custom arithmetic circuit. In each case, these representations are fed into a general-purpose (GPZKP) proof system for circuits or zkVMs. Each level of abstraction adds overhead and costs for the prover. Our approach is 'closer to the metal.' We do not use circuits or Plonk itself. Instead, we the same building blocks to build a custom PIOP argument for Zeeperio.

For online voting, where voters prove ballot validity, GPZKPs are studied, and we note a customized ZKP (which Zeeperio uses) is expected to be faster [19]. Closer to Zeeperio, another design for a succinct proofs use zk-*STARKs*, which do not have a trusted setup and are expected to have post-quantum security [18]. However, because this work uses GPZKPs, Zeeprio's custom PIOP approach can prove counted-as-cast for about 50,000 ballots in an hour (single core), compared with around 150 (single core) to 5000 (multicore) ballots.

2.3 Smart Contracts

Smart contracts are programs deployed on a blockchain, such as Ethereum. The blockchain's consensus mechanism ensures the function call executes correctly and updates the code is updated accordingly. In Zeeprio, a smart contract will accept an election proof and verify it, reporting whether it is correct or not. Since smart contracts can implement arbitrary code, they can support cryptographic operations, though this approach is usually prohibitively expensive. Ethereum offers cheaper 'pre-compiles' for pairing-based operations on the elliptic curve `alt-bn128`. On Ethereum's testnet, support for curve `bls12-381` has been staged for future deployment, as have KZG-specific operations over `bls12-381`, using the aforementioned SRS. Zeeperio on `alt-bn128` or `bls12-381`.

3 Zeeperio

Parameters. Let $\mathcal{B}$ be the number of printed ballots and $\mathcal{C}$ the number of candidates per ballot. Let $\mathcal{P} = \mathcal{BC}$ be the number of ballot positions. Let n be the padded column length (a power of two) with $n \geq \mathcal{P}$ (the need is explained

below). Let $\mathbb{F}_q$ be a finite field of size q. A ballot has ID $b \in \{0, \ldots, \mathcal{B}-1\}$ and occupies a *block*: $I_b = \{\, b\mathcal{C},\, b\mathcal{C}+1,\, \ldots,\, b\mathcal{C}+(\mathcal{C}-1)\,\} \subseteq \{0, \ldots, n-1\}$.

Walkthrough of a constraint. As shown in Figure 1, the mark column entries are all required to be either 0 or 1. With ZKPs, we call this a *constraint*. Zeeperio consists of 16 similar constraints that must all hold in any valid election table. Here, we present a complete argument for a single constraint (denoted c2.2) to demonstrate how a PIOP works. This constraint is adapted from [6].

The mark column can be viewed as a one-dimensional array $A_{\mathsf{mark}}[i]$ of length $\mathcal{P}$. The elements of the array are encoded as elements of $\mathbb{F}_q$, the exponent field of a pairing-friendly elliptic curve such as `alt-bn128` or `bls12-381`. Cryptographically, the array $A_{\mathsf{mark}}[i]$ could hold any integer in $[0, q-1]$, but the constraint requires it to be further restricted to $[0, 1]$ for a valid election table.

To prove the constraint, the prover (election authority) sends to the verifier (smart contract) a succinct (one group element) commitment (hiding and binding) to the entire mark column $A_{\mathsf{mark}}[i]$. Although vector commitments exist, in a PIOP the prover commits to $A_{\mathsf{mark}}[i]$ indirectly by first encoding the data from $A_{\mathsf{mark}}[i]$ into a univariate polynomial over $\mathbb{F}_q$ and then committing to this polynomial representation of $A_{\mathsf{mark}}[i]$ using a polynomial commitment scheme such as KZG.

There are several ways to encode vectors as polynomials, and we will focus on the method we use. The prover chooses a set of x-coordinates, such as $\{0, 1, 2, 3, \ldots\}$, called the evaluation domain $\mathcal{H}$. It then takes the data from $A_{\mathsf{mark}}[i]$ and forms a set of points $\{(0, A_{\mathsf{mark}}[0]), (1, A_{\mathsf{mark}}[1]), \ldots\}$. The prover interpolates a univariate polynomial through these points: $\mathsf{Mark}(X)$. It commits to $\mathsf{Mark}(X)$ using the polynomial commitment scheme and sends the commitment to the verifier.

Interpolation is computationally expensive for provers, so they use an optimization. The evaluation domain is arbitrary and can be any set as long as the prover and verifier agree. Let $\omega \in \mathbb{F}_q$ generate a multiplicative subgroup of size n. Then set $H = \{\omega^0, \omega^1, \ldots, \omega^{n-1}\} \subseteq \mathbb{F}_q$. Doing so offers three advantages and one drawback. Interpolating a polynomial over this domain can be accomplished in $\mathcal{O}(n \log n)$ instead of $\mathcal{O}(n^2)$. Traversing the domain is easy: the next index is the current index multiplied by ω, and because it is closed under multiplication, the 'next' element after the last element ω^{n-1} wraps back to the first element $\omega^{n-1} \cdot \omega = \omega^n = \omega^0$. Finally, a polynomial that is 0 at each $X \in H$ has a simple closed form: $\mathsf{Z}_H(X) = X^n - 1$. The drawback is that the prover cannot use arrays of arbitrary length $\mathcal{P}$ and expect that an order n subgroup will just exist. The subgroups are based on the parameters of the elliptic curve, and curves like `alt-bn128` and `bls12-381` are designed to offer subgroups of sizes $\{2, 2^2, 2^3, 2^4, \ldots\}$ up to 2^{12} for `alt-bn128` and 2^{32} for `bls12-381`.

In Zeeprio, if we have a column of length $\mathcal{P}$, we will zero-pad it to the nearest power of 2. Thus, n is the padded column length (a power of two) with $n \geq \mathcal{P}$. For this example constraint (the mark column is binary), the presence of zero padding does not affect the constraint. However, for other constraints, we will have to explicitly handle the padding separately from the election table data.

At this point, the verifier holds a polynomial commitment to $\mathsf{Mark}(X)$, which we call K_{Mark}. How does the prover demonstrate that $\mathsf{Mark}(X)$ is binary? The prover constructs another polynomial as follows: $V(X) := \mathsf{Mark}(X)\big(\mathsf{Mark}(X) - 1\big)$. The prover commits to $V(X)$ and sends K_V to the verifier. If $\mathsf{Mark}(X)$ is binary for all $X \in H$, then $V(X)$ is zero for all $X \in H$. If $\mathsf{Mark}(X)$ is not binary for at least one $X \in H$, then $V(X)$ is non-zero at that X. We have converted the binary constraint on $\mathsf{Mark}(X)$ into the following two checks: (1) $V(X)$ is correctly constructed from $\mathsf{Mark}(X)$, and (2) $V(X)$ is zero for all $X \in H$. A polynomial that is zero for all $X \in H$ is called a *vanishing polynomial* (on H). To prove that $V(X)$ is correctly constructed from $\mathsf{Mark}(X)$, the verifier has K_{Mark} and K_V. The verifier will (or the prover, using the strong Fiat-Shamir heuristic) choose a random evaluation point $\zeta \notin H$. Using the polynomial commitment scheme, the prover will open the polynomials at ζ, sending $V(\zeta)$, $\mathsf{Mark}(\zeta)$, and the opening proofs for each to the verifier. The verifier will check the opening proofs and check that $V(\zeta) := \mathsf{Mark}(\zeta)\big(\mathsf{Mark}(\zeta) - 1\big)$. Just because the relation holds at ζ, what does that say about it holding for each $X \in H$? The answer is that it holds with overwhelming probability due to the Schwartz-Zippel lemma, which we do not review here.

Next, the prover will demonstrate that $V(X)$ vanishes on H. Recall that $\mathsf{Z}_H(X) = X^n - 1$ is a vanishing polynomial on H. The prover constructs another polynomial $Q(X) := V(X)/\mathsf{Z}_H(X)$, commits to $Q(X)$, and sends the commitment K_Q to the verifier. If and only if $V(X)$ is zero at every point of H, $\mathsf{Z}_H(X)$ divides $V(X)$ exactly, yielding a proper polynomial $Q(X)$; otherwise, $Q(X)$ is a rational function not reducible to a polynomial. $Q(X) := V(X)/\mathsf{Z}_H(X)$ can be rearranged as $V(X) - Q(X) \cdot (X^n - 1) = 0$. Using ζ from above, the prover sends $Q(\zeta)$ and its opening proof. The verifier checks $V(\zeta) - Q(\zeta) \cdot (\zeta^n - 1) = 0$. By Schwartz–Zippel, if this equality holds at a uniformly random $\zeta \notin H$, then, except with negligible probability, it holds as a polynomial identity, which implies that $V(X)$ vanishes on H. □

3.1 Zeeperio Constraints

Zeeperio consists of 16 constraints. Each constraint follows a similar pattern to the example constraint: a set of polynomials is committed to such that a relationship between them will be a vanishing polynomial if and only if the constraint is true. Once the constraint is converted into a vanishing polynomial, proving the vanishing polynomial is constructed correctly and that it actually vanishes is identical to the process used above.

In fact, given a set of vanishing polynomials $\{V_1(X), V_2(X), \ldots, V_m(X)\}$ and their corresponding quotient polynomials, the checks can be batched into a single check. The prover will form a 'super-polynomial' $\mathsf{Batch}(X)$ whose coefficients are the vanishing polynomials:

$$\mathsf{Batch}(Y) = \sum_{i=1}^{m} Y^i \Big(V_i(X) - Q_i(X)\,\mathsf{Z}_H(X)\Big).$$

Table 1. A set of polynomials that can be computed before the beginning of the election. $\mathsf{Sel}_b(X)$ is the one exception, it will not be precomputed but rather computed during a receipt check just for the one ballot b that the voter wants to check.

Description	**Polynomial**
Polynomial that is zero on H	$\mathsf{Z}_H(X) = X^n - 1$
Mask that is 0 on all of H except possibly at $X = \omega^0$	$\mathsf{Mask}_0(X) = \dfrac{\mathsf{Z}_H(X)}{X - \omega^0}$
Mask that is 0 on all of H except possibly at $X = \omega^{n-1}$	$\mathsf{Mask}_{n-1}(X) = \dfrac{\mathsf{Z}_H(X)}{X - \omega^{n-1}}$
Polynomial that is 0 up to padding $[0, \mathcal{P}-1]$	$\mathsf{Z}_{\mathsf{head}}(X) = \prod_{i=0}^{\mathcal{P}-1} \left(X - \omega^i\right)$
Polynomial that is 0 for the last $\mathcal{C}$ rows (zeros at $\omega^{n-1}, \ldots, \omega^{n-\mathcal{C}}$)	$\mathsf{Mask}_{\mathsf{tail}}(X) = \prod_{t=0}^{\mathcal{C}-1} \left(X - \omega^{n-1-t}\right)$
Lagrange basis over H used below	$L_i(X) = \prod_{\substack{0 \le j < n \\ j \ne i}} \dfrac{X - \omega^j}{\omega^i - \omega^j}$
Polynomial with a 1 at the start of every ballot (indices $i \equiv 0 \pmod{\mathcal{C}}$ up to padding) and 0 elsewhere	$\mathsf{Sel}_{\mathsf{blk}}(X) = \sum_{j=0}^{\lfloor n/\mathcal{C} \rfloor - 1} L_{j\mathcal{C}}(X)$
Polynomial that is 1 for all rows belonging to ballot-b ($I_b = \{b\mathcal{C}, \ldots, b\mathcal{C}+\mathcal{C}-1\}$) and 0 elsewhere	$\mathsf{Sel}_b(X) = \sum_{k=0}^{\mathcal{C}-1} L_{b\mathcal{C}+k}(X)$

A polynomial with all coefficients 0 is a zero polynomial. Therefore, the verifier (or prover via strong Fiat-Shamir) will choose a random $\zeta \in \mathbb{F}_q \notin H$ as before and set $X = \zeta$, which makes every coefficient 0 with overwhelming probability. To check whether every coefficient is 0, it will also choose a random $\alpha \in \mathbb{F}_q$ and set $Y = \alpha$. If every $V_i(X)$ vanishes, then $\mathsf{Batch}(Y)$ is a zero polynomial and:

$$\mathsf{Batch}(\alpha) = \sum_{i=1}^{m} \alpha^i \Big(V_i(\zeta) - Q_i(\zeta)\, \mathsf{Z}_H(\zeta) \Big) = 0.$$

3.2 Public Polynomials

Zeeperio uses several public helper polynomials that depend only on the election 'shape' (e.g., n, $\mathcal{B}$, $\mathcal{C}$, and hence $\mathcal{P} = \mathcal{B}\mathcal{C}$), not on any voter choices or the contents of the committed columns. They are enumerated in Table 1. Some of these helpers are linear or superlinear in n to construct (for example, explicit products or explicit sums of Lagrange basis polynomials). This does not undermine Zeeprio's claims of succinctness: the pre-computation is offline, public, and independent of the witness. In systems like Plonk, these polynomials are considered part of the 'verifier's key.' In practice, election administrators can standardize on

a small set of template sizes, and these polynomials can be pre-computed once for each election template. If an election uses fewer ballots or candidates than the template's maximum, the remaining rows are constrained to be empty: extra ballots are unused, and extra candidates receive zero votes, which is reflected directly in the published tally.

3.3 Election Specific Polynomials

The Eperio tables for an election consist of three (padded) columns, which are encoded as polynomials evaluated over H. We denote these polynomials $\mathsf{Code}(X)$, $\mathsf{Audit}(X)$, and $\mathsf{Mark}(X)$. Each index maps to a unique ballot ID and candidate, and the public template provides this mapping. The columns are sorted by ballot ID, then by candidate (see Figure 1). The final tally is a vector $\mathsf{Tally}[0..\mathcal{C}-1] \in \mathbb{F}_q^{\mathcal{C}}$.

Across all constraints, we will use an additional 5 'helper' polynomials. We will introduce them as needed. They consist of two (additive, suffix) accumulator polynomials: $\mathsf{Acc}_{\mathsf{Audit}}(X)$ and $\mathsf{Acc}_{\mathsf{Mark}}(X)$, where $\mathsf{Acc}_{\mathsf{Audit}}(\omega^i) = \sum_{t=i}^{n-1} \mathsf{Audit}(\omega^t)$; likewise for $\mathsf{Mark}(X)$. We also define a 'strided' accumulator $\mathsf{Acc}_{\mathsf{Vote}}(X)$, which we will describe in constraint 4. Finally, we define two block sum helper polynomials: $\mathsf{BlkSum}_{\mathsf{Audit}}(X) = \sum_{k=0}^{\mathcal{C}-1} \mathsf{Audit}(\omega^k X)$ and $\mathsf{BlkSum}_{\mathsf{Mark}}(X) = \sum_{k=0}^{\mathcal{C}-1} \mathsf{Mark}(\omega^k X)$.

3.4 Audit Constraints

Due to limited space in the paper, we cannot fully explain each of the 16 constraints. Instead we provide some intuition for them. The audit constraints are:

$$
\begin{aligned}
V_{c1.1}(X) &:= \mathsf{Audit}(X)\,\mathsf{Z}_{\mathsf{head}}(X) \\
V_{c1.2}(X) &:= \mathsf{Audit}(X)\big(\mathsf{Audit}(X)-1\big) \\
V_{c1.3}(X) &:= \big(\mathsf{Sel}_{\mathsf{blk}}(X)\mathsf{BlkSum}_{\mathsf{Audit}}(X)\big)\big(\mathsf{Sel}_{\mathsf{blk}}(X)\mathsf{BlkSum}_{\mathsf{Audit}}(X)-\mathcal{C}\big) \\
V_{c1.4}(X) &:= \big(\mathsf{Acc}_{\mathsf{Audit}}(X)-\mathsf{Audit}(X)+\mathsf{Acc}_{\mathsf{Audit}}(\omega X)\big)\cdot(X-\omega^{n-1}) \\
V_{c1.5}(X) &:= \big(\mathsf{Acc}_{\mathsf{Audit}}(X)-\mathsf{Audit}(X)\big)\cdot\mathsf{Mask}_{n-1}(X) \\
V_{c1.6}(X) &:= \big(\mathsf{Acc}_{\mathsf{Audit}}(X)-\mathsf{Sum}_{\mathsf{Audit}\times\mathcal{C}}\big)\cdot\mathsf{Mask}_0(X)
\end{aligned}
$$

The audit column is well-defined for an election if all padding bits are 0 (c1.1) and the column is otherwise binary (c1.2). Each ballot spans $\mathcal{C}$ positions, so the audit column should contain a contiguous block of $\mathcal{C}$ ones for an audited ballot and a block of $\mathcal{C}$ zeros for unaudited ballots. The mechanism computes a block-sum over each ballot, $\mathsf{BlkSum}_{\mathsf{Audit}}(X) = \sum_{k=0}^{\mathcal{C}-1} \mathsf{Audit}(\omega^k X)$, then restricts attention to ballot-start indices using $\mathsf{Sel}_{\mathsf{blk}}(X)$. For a well-formed audit column, the filtered block-sum is always either 0 (not audited) or $\mathcal{C}$ (audited), and never an intermediate value. After choosing ζ, in order to check $\mathsf{BlkSum}_{\mathsf{Audit}}(X)$ is well-formed, the verifier requires $\mathcal{C}$ opened points from $\mathsf{Audit}(\omega^0\zeta)$ to $\mathsf{Audit}(\omega^{\mathcal{C}-1}\zeta)$, so this check is linear in $\mathcal{C}$ (but more importantly, constant in $\mathcal{P}$ and n).

The EA will assert the number of print audits in the election as the integer $\mathsf{Sum}_{\mathsf{A}}$. Since each print audit ballot contains a block of $\mathcal{C}$ 1's, the number of 1's

in $A(X)$ must be $\mathsf{Sum}_{\mathsf{A}\times\mathcal{C}} = \mathsf{Sum}_{\mathsf{A}} \cdot \mathcal{C}$. To reason about the sum across a polynomial on H, many protocols use accumulator polynomials [15, 6]. $\mathsf{Acc}_{\mathsf{Audit}}(X)$ is intended to hold a running suffix-sum of the audit column down the domain order (including padding, which is forced to be 0 by c1.1):

$$\begin{aligned} \mathsf{Acc}_{\mathsf{Audit}}(\omega^i) &:= \mathsf{Audit}(\omega^i) + \mathsf{Acc}_{\mathsf{Audit}}(\omega^{i+1}) \quad \text{for } i = 0, 1, \ldots, n-2, \\ \mathsf{Acc}_{\mathsf{Audit}}(\omega^{n-1}) &:= \mathsf{Audit}(\omega^{n-1}). \end{aligned}$$

Constraint c1.4 and c1.5 check each line of this recurrence respectively, guaranteeing that $\mathsf{Acc}_{\mathsf{Audit}}(X)$ is formed correctly. The first value in $\mathsf{Acc}_{\mathsf{Audit}}(X)$ holds the sum, so c1.6 ensures this value is equal to $\mathsf{Sum}_{\mathsf{A}\times\mathcal{C}}$.

3.5 Mark Constraints

The constraints on the mark column parallel the constraints on the audit column:

$$\begin{aligned} V_{c2.1}(X) &:= \mathsf{Mark}(X)\,\mathsf{Z}_{\mathsf{head}}(X) \\ V_{c2.2}(X) &:= \mathsf{Mark}(X)\big(\mathsf{Mark}(X) - 1\big) \\ V_{c2.3}(X) &:= \big(\mathsf{Sel}_{\mathsf{blk}}(X)\mathsf{BlkSum}_{\mathsf{Mark}}(X)\big)\big(\mathsf{Sel}_{\mathsf{blk}}(X)\mathsf{BlkSum}_{\mathsf{Mark}}(X) - 1\big) \\ V_{c2.4}(X) &:= \big(\mathsf{Acc}_{\mathsf{Mark}}(X) - \mathsf{Mark}(X) + \mathsf{Acc}_{\mathsf{Mark}}(\omega X)\big) \cdot (X - \omega^{n-1}) \\ V_{c2.5}(X) &:= \big(\mathsf{Acc}_{\mathsf{Mark}}(X) - \mathsf{Mark}(X)\big) \cdot \mathsf{Mask}_{n-1}(X) \\ V_{c2.6}(X) &:= \big(\mathsf{Acc}_{\mathsf{Mark}}(X) - \mathsf{Sum}_{\mathsf{Mark}}\big) \cdot \mathsf{Mask}_0(X) \end{aligned}$$

The constraints are as follows: padding elements are zero (c2.1), column elements are binary (c2.2), and the total number of votes matches the sum announced by the election authority $\mathsf{Sum}_{\mathsf{Mark}}$ (c2.4–c2.5). The main difference is constraint c2.3. For each ballot in the audit column (previous section), the ballot's block is either all 0 or all 1, summing to either 0 or $\mathcal{C}$. For the vote column (this section), each ballot block is either all 0 (not voted) or contains a single 1 (voted). Since elements are binary (c2.2), the EA can demonstrate that the sum of each ballot block is either 0 or 1, by c2.3. We assume that over-voted ballots are rejected by the scanner or automatically converted into print-audit ballots.

3.6 Cross-Column Constraints

A final constraint demonstrates that no ballot is both voted and audited:

$$V_{c3.1}(X) := \mathsf{Audit}(X)\,\mathsf{Mark}(X)$$

This constraint is straightforward: an unvoted ballot will have all 0's in its ballot block, while an unaudited ballot will have the same. Therefore, the Hadamard product (element-wise multiplication) of the audit column with the mark column

will be 0 in every element if it is properly formed. In the malicious case, if a ballot is both voted and audited, at least one element of the ballot block will have 1 in both columns, and the product of the columns will not be 0 at that element; thus, $V_{c3.1}(X)$ will not vanish.

3.7 Tally Constraints

The tally constraints are as follows:

$$
\begin{aligned}
V_{c4.1}(X) &:= \big(\mathsf{Acc}_{\mathsf{Vote}}(X) - \mathsf{Mark}(X) + \mathsf{Acc}_{\mathsf{Vote}}(\omega^{\mathcal{C}} X)\big) \cdot \mathsf{Mask}_{\mathsf{tail}}(X) \\
V_{c4.2}(X) &:= \big(\mathsf{Acc}_{\mathsf{Vote}}(X) - \mathsf{Mark}(X)\big) \cdot \frac{\mathsf{Z}_H(X)}{\mathsf{Mask}_{\mathsf{tail}}(X)} \\
V_{c4.3,j}(X) &:= \big(\mathsf{Acc}_{\mathsf{Vote}}(X) - \mathsf{Tally}[j]\big) \cdot \frac{\mathsf{Z}_H(X)}{X - \omega^j}
\end{aligned}
$$

The EA will announce the tally as a list of vote totals for each of the $\mathcal{C}$ candidates: $\mathsf{Tally}[j]$ for $j \in \{0, \ldots, \mathcal{C} - 1\}$. The marks for candidate i will be in every i-th element of $\mathsf{Mark}(X)$ over H. The EA will build an additive, suffix accumulator $\mathsf{Acc}_{\mathsf{Vote}}(X)$ however it will be *strided*:

$$
\mathsf{Acc}_{\mathsf{Vote}}(\omega^i) := \sum_{t=0}^{\lfloor \frac{n-1-i}{\mathcal{C}} \rfloor} \mathsf{Mark}\big(\omega^{i+t\mathcal{C}}\big), \qquad i \in \{0, \ldots, n-1\}.
$$

The result is that $\mathsf{AccVote}(\omega^c)$ is equal the total number of marks appearing at indices congruent to c mod $\mathcal{C}$ across all ballots (and padding). Note that there is no assumption that $c|n$; the sums for some candidates may be one element longer than those for others. Constraint c4.1 demonstrates that the accumulator is built correctly, assuming it starts with the right values. Constraint c4.2 demonstrates that the starting values are correct. Constraint c4.3 shows that the published tally equals the sum of each candidate's votes. Constraint c4.3 is run $\mathcal{C}$ times, once for each candidate's total votes, which will be contained in the first $\mathcal{C}$ elements of $\mathsf{AccVote}(X)$ on H.

Security Properties. A full security analysis of Zeeperio is deferred to the extended thesis version[1]. However, Zeeperio's security leverages existing literature. Eperio has already been proven secure in both cryptographic [14] and formal-methods [3], establishing end-to-end verifiability. Modern PIOP-based SNARKs achieve security by reducing algebraic constraints to vanishing-polynomial checks [15]. Thus, security reduces to a straightforward question: do our constraints faithfully encode the same validity conditions as Eperio? Since they do, completeness and soundness follow directly from the security of the KZG commitment scheme and the PIOP model.

[1] Link withheld for anonymity.

4 Voter Checks

Eperio, like many voting systems, splits the E2E-V proof into two parts: a universal verifiable proof that the tally is correct, and a voter-verifiable proof that receipts are correct. All constraints from Section 3 correspond to the former proof, whereas in this section we describe the voter-verifiable checks.

4.1 Print Audit

The voter provides ballot ID b, which determines the public index set $I_b = b\mathcal{C}, \ldots, b\mathcal{C} + \mathcal{C} - 1$. The EA opens the committed polynomials $\mathsf{Audit}(X)$, $\mathsf{Mark}(X)$, and $\mathsf{Code}(X)$ at the $\mathcal{C}$ points ${\omega^i}_{i \in I_b}$ and returns those evaluations with opening proofs (for KZG, these can be batched in a single multipoint opening proof).

4.2 Receipt Check

At a first glance, it might seem the EA can simply open $\mathsf{Mark}(X)$ and $\mathsf{Code}(X)$ for the voter's ballot ID b. However, since these polynomials are ordered by candidate, the index of the mark indicates the candidate the voter voted for. We analyzed three PIOP-friendly approaches to this, but for space considerations, we present only the best: using a permutation (briefly, the excluded alternatives are lookup arguments and a secret selector vector with summation).

First, the EA defines a public *ballot-ID column* $\mathsf{BallotID}(X)$ that encodes the ballot identifier on every ballot position: for each ballot $b \in \{0, \ldots, \mathcal{B} - 1\}$ and each within-ballot offset $k \in \{0, \ldots, \mathcal{C} - 1\}$, we set $\mathsf{BallotID}(\omega^{b\mathcal{C}+k}) = b$, and we set $\mathsf{BallotID}(\omega^i) = 0$ on all padding indices $i \geq \mathcal{P}$. Next, the EA samples a secret permutation π over $\{0, \ldots, n - 1\}$ and uses it to define permuted columns by $\mathsf{F}^\pi(\omega^i) = \mathsf{F}(\omega^{\pi(i)})$. The EA then commits once (pre-election or once-per-election) to the shuffled columns $\mathsf{BallotID}^\pi(X)$, $\mathsf{Code}^\pi(X)$, and $\mathsf{Mark}^\pi(X)$, and proves once that each shuffled column is a permutation of its corresponding source column under the *same* secret permutation π. The permutation argument comes straight from the literature [15].[2]

For a per-receipt query, the voter provides ballot ID b. The EA finds, privately, an index t such that $\mathsf{BallotID}^\pi(\omega^t) = b$ and $\mathsf{Mark}^\pi(\omega^t) = 1$, and it returns the receipt code $c := \mathsf{Code}^\pi(\omega^t)$ together with a batched opening proof at the single point ω^t for the three committed polynomials $\mathsf{BallotID}^\pi(X)$, $\mathsf{Mark}^\pi(X)$, and $\mathsf{Code}^\pi(X)$. Revealing index t is safe because π is secret, so t does not reveal the candidate-order position of the mark in the original (unshuffled) ballot block.

4.3 Dispute Resolution

If a receipt check returns a wrong confirmation code, it is possible that the EA switched the voter's vote to another candidate. However, the voter has proof

[2] The only minor difference is that we are shuffling the random linear combination of three polynomials instead of two polynomials under the same permutation.

of this: they know a different confirmation code that appears on the ballot. By contrast, a voter who wants to discredit an election result might claim spuriously that their confirmation code is incorrect (even though it is); however, they will not know a different valid confirmation on their ballot (beyond guessing) [11].

In a dispute, the voter provides a ballot ID b and a disputed confirmation code $c \in \mathbb{F}_q$. The EA's goal is to prove *non-membership*: that c does not appear anywhere among the $\mathcal{C}$ confirmation codes printed on ballot b, without revealing any of those codes. Let $I_b = \{b\mathcal{C}, \ldots, b\mathcal{C} + (\mathcal{C} - 1)\}$ denote the (public) index set of ballot b, and let $\mathsf{Mask}^{(b)}_{\mathsf{ballot}}(X)$ be the public 0/1 mask that is 1 on I_b and 0 elsewhere on H. If c is a non-member, then $(\mathsf{Code}(\omega^i) - c)$ will be non-zero on each element in I_b. If and only if it is non-zero, an inverse element will exist such that $(\mathsf{Code}(\omega^i) - c) \cdot \mathsf{Inv}_b(\omega^i) = 1$. The EA introduces an auxiliary polynomial of asserted inverses $\mathsf{Inv}_b(X)$ on I_b and the constraint reduces the vanishing polynomial:

If a receipt check returns a wrong confirmation code, it is possible that the EA switched the voter's vote to another candidate. However, the voter has proof of this: they know a different confirmation code that appears on the ballot. By contrast, a voter who wants to discredit an election result might claim spuriously that their confirmation code is incorrect (even though it is); however, they will not know a different valid confirmation on their ballot (beyond guessing) [11].

In a dispute, the voter provides a ballot ID b and a disputed confirmation code $c \in \mathbb{F}_q$. The EA's goal is to prove *non-membership*: that c does not appear anywhere among the $\mathcal{C}$ confirmation codes printed on ballot b, without revealing any of those codes. Let $I_b = \{b\mathcal{C}, \ldots, b\mathcal{C} + (\mathcal{C} - 1)\}$ denote the (public) index set of ballot b, and let $\mathsf{Mask}^{(b)}_{\mathsf{ballot}}(X)$ be the public 0/1 mask that is 1 on I_b and 0 elsewhere on H. If c is a non-member, then $(\mathsf{Code}(\omega^i) - c)$ will be non-zero on each element in I_b. If and only if it is non-zero, an inverse element will exist such that $(\mathsf{Code}(\omega^i) - c) \cdot \mathsf{Inv}_b(\omega^i) = 1$. The EA introduces an auxiliary polynomial of asserted inverses $\mathsf{Inv}_b(X)$ on I_b and the constraint reduces the vanishing polynomial:

$$V_{\mathsf{dispute}}(X) \;:=\; \big((\mathsf{Code}(X) - c) \cdot \mathsf{Inv}_b(X) - 1\big) \cdot \mathsf{Mask}^{(b)}_{\mathsf{ballot}}(X)$$

If c does appear on the ballot, say at position $\kappa \in I_b$, the EA will not be able to put a value in $\mathsf{Inv}_b(\kappa)$ that makes $V_{\mathsf{dispute}}(X)$ vanish.

5 Implementation and Performance

Zeeperio was implemented and tests were run on an Apple Macbook Pro (M3 Pro, 18 GB RAM). Table 2 shows the performance for each of the three proofs.

Off-chain Prover. The prover is implemented in Rust and uses the *arkworks* toolkit for BN254 pairings. The instance is launched with a CLI that generates all three proofs described in Sections 3 and 4. It commits to all polynomials using the *hiding* variant of KZG, where each commitment includes a blinding term derived

Table 2. Zeeperio benchmark with $\mathcal{B} = 100{,}000$ ballots, $\mathcal{C} = 5$ candidates, $\mathcal{P} = \mathcal{BC} = 500,000$ positions, padding to 2^20, and 10% of ballots print audited. All measurements are from a single-core execution on standard hardware (M3 Pro).

Metric	Main	Inclusion	Receipt
Proof generation (s)	8,709	3,849	3,823
Commitments (#)	24	13	6
Opening points (#)	6	2	2
Calldata size (bytes)	8416	4128	2688
Transaction fee (Gas)[3]	2,429,116	1,029,029	727,118

from the SRS. The prover also uses *off-domain masking* where it randomizes witness polynomials with additional masking to prevent any information leakage. To maximize cost efficiency, the prover batches all polynomial evaluations using a random scalar γ into a single KZG opening proof. This allows the verifier to check all evaluations simultaneously with a single pairing check. Finally, the proofs are exported to calldata using Ethereum ABI encoding.

Onchain Verifier. The verifier smart contract is implemented in Solidity using the `Foundry` project setup. The contract takes an ABI-encoded proof containing the public inputs, polynomial commitments and opening proofs, and verifies them on-chain using the BN254 pairing precompiles. The verification logic is split into three functions: `verifyMain`, `verifyInclusion`, and `verifyReceipt`, corresponding to the three proofs described in Sections 3 and 4. It should be noted that both prover and verifier recompute the same challenges, so they need to share the same serialization rules.

6 Concluding Remarks

Zeeperio is a succinct proof of an Eperio-encoded election that can be autonomously verified by a smart contract. Even if the reader does not buy the notion of realcryptik, that not hiding raw ballots from the EA can be a stepping stone to full E2E-V elections, Zeeperio is still useful as a drop-in (or complementary) mechanism for standard Eperio. Eperio (like Scantegrity and Punchscan) makes a 'blackbox assumption' for ballot privacy that can be extended to Zeeperio. This leads to a natural adoption pathway for vendors and election officials: Zeeprio introduces an initial verifiability component, with automated argument checking via smart contracts, as a first step. Stronger ballot secrecy guarantees can later be integrated with TEE's for full E2E-V.

References

1. Adida, B.: Helios: web-based open-audit voting. In: USENIX Security Symposium. pp. 335–348 (2008)

[3] At an ETH price of \$3,000 USD, gas price of 4.5 Gwei, and using 1 Gwei = 10^{-9} ETH, these transaction fees are approximately $\sim$ \$34, $\sim$ \$14, and $\sim$ \$10 respectively.

2. Adida, B., Marneffe, O.d., Pereira, O., Quisquater, J.J.: Electing a university president using open-audit voting: Analysis of real-world use of Helios. In: EVT/WOTE (2009)
3. Basin, D., Dreier, J., Giampietro, S., Radomirović, S.: Verifying table-based elections. In: ACM CCS (2021)
4. Bell, S., Benaloh, J., Byrne, M.D., DeBeauvoir, D., Eakin, B., Kortum, P., McBurnett, N., Pereira, O., Stark, P.B., Wallach, D.S., Winn, M.: {STAR-Vote}: A secure, transparent, auditable, and reliable voting system. In: EVT/WOTE (2013)
5. Bernhard, M., McDonald, A., Meng, H., Hwa, J., Bajaj, N., Chang, K.: Can voters detect malicious manipulation of ballot marking devices? In: IEEE Symposium on Security and Privacy (2020)
6. Boneh, D., Fisch, B., Gabizon, A., Williamson, Z.: A simple range proof from polynomial commitments (2019), https://hackmd.io/@dabo/B1U4kx8XI
7. Brassard, G., Chaum, D., Crépeau, C.: Minimum disclosure proofs of knowledge. J. Comput. Syst. Sci. **37**(2), 156–189 (1988)
8. Bünz, B., Fisch, B., Szepieniec, A.: Transparent snarks from dark compilers. In: EUROCRYPT (2020)
9. Carback, R.T., Chaum, D., Clark, J., Conway, J., Essex, A., Hernson, P.S., Mayberry, T., Popoveniuc, S., Rivest, R.L., Shen, E., Sherman, A.T., Vora, P.L.: Scantegrity II municipal election at Takoma Park: the first E2E binding governmental election with ballot privacy. In: USENIX Security Symposium (2010)
10. Cardillo, A., Akinyokun, N., Essex, A.: Online voting in ontario municipal elections: A conflict of legal principles and technology? International Joint Conference on Electronic Voting **11759**, 67–82 (2019)
11. Chaum, D., Carback, R., Clark, J., Essex, A., Popoveniuc, S., Rivest, R.L., Ryan, P.Y.A., Shen, E., Sherman, A.T.: Scantegrity II: end-to-end verifiability for optical scan election systems using invisible ink confirmation codes. In: EVT (2008)
12. Chiesa, A., Hu, Y., Maller, M., Mishra, P., Vesely, N., Ward, N.: Marlin: Preprocessing zksnarks with universal and updatable srs. In: EUROCRYPT (2020)
13. Culnane, C., Ryan, P.Y., Schneider, S., Teague, V.: vvote: a verifiable voting system. ACM Transactions on Information and System Security (TISSEC) **18**(1), 1–30 (2015)
14. Essex, A., Clark, J., Hengartner, U., Adams, C.: Eperio: Mitigating technical complexity in cryptographic election verification. In: EVT/WOTE (2010)
15. Gabizon, A., Williamson, Z.J., Ciobotaru, O.: Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive (2019)
16. Goldreich, O., Micali, S., Wigderson, A.: How to prove all np statements in zero-knowledge and a methodology of cryptographic protocol design. In: CRYPTO (1986)
17. Hao, F., Ryan, P.Y.: Real-world electronic voting: Design, analysis and deployment. CRC Press (2016)
18. Harrison, M., Haines, T.: On the applicability of starks to counted-as-collected verification in existing homomorphic e-voting systems. In: VOTING (2025)
19. Huber, N., Kuesters, R., Liedtke, J., Rausch, D.: ZK-SNARKs for ballot validity: A feasibility study. In: E-VOTE-ID (2024)
20. Kate, A., Zaverucha, G.M., Goldberg, I.: Constant-size commitments to polynomials and their applications. In: ASIACRYPT. pp. 177–194. Springer (2010)
21. Maller, M., Bowe, S., Kohlweiss, M., Meiklejohn, S.: Sonic: Zero-knowledge SNARKs from linear-size universal and updatable structured reference strings. In: CCS (2019)